

Antes de Iniciar

Antes de entendermos o que é HTML5, é importante saber o que significa uma linguagem de marcação.

Uma linguagem de marcação é usada para estruturar, organizar e descrever o conteúdo de um documento. Ela não é uma linguagem de programação — como JavaScript ou Python — mas sim uma forma de dizer ao navegador como o conteúdo deve ser exibido. Por exemplo, podemos usar marcações (ou 'tags') para indicar que um texto é um título, um parágrafo, uma imagem ou um link.

No contexto da web, a principal linguagem de marcação é o HTML (HyperText Markup Language), que serve para criar páginas da internet. Com HTML, podemos montar a estrutura visual de um site e organizar seu conteúdo de forma clara e lógica, tanto para os usuários quanto para os navegadores e mecanismos de busca. Compreender HTML é o primeiro passo essencial para quem deseja desenvolver para a web.

Encontro 1 – Introdução ao HTML5 e Estrutura Básica

1. O que é HTML5?

HTML5 é a versão mais atual da linguagem de marcação usada para criar páginas web. Com ele, podemos estruturar o conteúdo de uma página utilizando 'tags' (etiquetas), que informam ao navegador como aquele conteúdo deve ser apresentado. Por exemplo, podemos indicar o que é um título, um parágrafo, uma imagem, um link ou até mesmo um formulário de contato. O HTML5 trouxe melhorias importantes, como o suporte direto a vídeos e áudios, novas tags semânticas e melhor integração com JavaScript, tornando as páginas mais interativas e acessíveis.

2. Estrutura Básica de um Documento HTML5

A estrutura básica de uma página HTML5 é como o esqueleto de uma casa: define a base onde tudo será construído. Ela inclui a declaração do tipo de documento (DOCTYPE), a tag raiz <html>, e as seções <head> (cabeçalho invisível) e <body> (conteúdo visível). Dentro do <head>, colocamos informações como o título da página e a codificação de caracteres. No <body>, inserimos todo o conteúdo que o usuário verá na tela. É fundamental começar com essa estrutura para garantir que o navegador interprete corretamente o conteúdo.

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8">
    <title>Título da Página</title>
  </head>
  <body>
    <!-- Conteúdo da página -->
  </body>
</html>
```

3. Explicação dos Principais Elementos

<!DOCTYPE html>: Essa linha aparece no início do documento e informa que estamos usando a versão HTML5.

<html>: É o elemento raiz. Tudo dentro da página deve estar contido entre **<html>** e **</html>**.

<head>: Aqui vão as configurações da página, como o título (que aparece na aba do navegador), definição de codificação e links de estilo.

<title>: Define o texto que será exibido na aba do navegador.

<body>: Tudo o que aparece na tela do navegador vai dentro do **<body>**: textos, imagens, vídeos, botões, etc.

- **<html>:** Elemento raiz da página.
- **<head>:** Contém metadados, como título, codificação e links de estilo.
- **<title>:** Define o título da aba do navegador.
- **<body>:** Contém todo o conteúdo visível da página.

4. Elementos de Cabeçalho, Título e Rodapé

O HTML5 introduziu elementos semânticos que ajudam a organizar melhor o código e a torná-lo mais compreensível tanto para desenvolvedores quanto para mecanismos de busca e tecnologias assistivas. Os principais são:

- **<header>:** representa o cabeçalho da página, geralmente contendo logotipo, título e menu de navegação.

- **<footer>:** representa o rodapé da página, com informações complementares como direitos autorais, contatos ou links úteis.

- **<h1>** a **<h6>:** são usadas para títulos e subtítulos. O **<h1>** é o mais importante e deve aparecer uma única vez na página, enquanto os demais podem ser usados em ordem hierárquica.

- **<header>:** Define o cabeçalho da página ou de uma seção.

- <footer>: Define o rodapé da página ou de uma seção.
- <h1> a <h6>: Representam títulos, sendo <h1> o mais importante.

5. Exemplo Prático

Veja abaixo um exemplo completo de uma página HTML estruturada corretamente. Este exemplo usa os principais elementos aprendidos até aqui.

É importante que você digite esse código manualmente para fixar a estrutura:

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <title>Minha Primeira Página</title>
</head>
<body>
  <header>
    <h1>Curso de Desenvolvedor Web</h1>
  </header>
  <h2>Encontro 1 - HTML5 Básico</h2>
  <h3>Primeiros Passos</h3>
  <footer>
    <p>© 2025 - Curso EAD</p>
  </footer>
</body>
</html>
```

6. Boas Práticas na Estruturação de um Arquivo HTML

Declare corretamente o tipo de documento

Todo arquivo HTML deve começar com a declaração <!DOCTYPE html>. Isso informa ao navegador que se trata de um documento HTML5, permitindo que ele interprete e exiba corretamente os elementos. Ignorar essa declaração pode resultar em comportamentos inesperados.

Use indentação para organizar o código

A indentação (recuo) é fundamental para tornar o código mais legível. Mesmo que o navegador ignore espaços em branco, para o programador é essencial manter uma organização clara dos elementos aninhados. O padrão é usar dois ou quatro espaços para cada nível de profundidade.

Organize o conteúdo com clareza

Utilize corretamente as seções do documento: `<head>` para metadados e configurações, e `<body>` para o conteúdo visual da página. Evite misturar responsabilidades. Coloque links para arquivos CSS no `<head>`, e scripts JavaScript ao final do `<body>`, sempre que possível.

Utilize elementos semânticos

HTML5 trouxe elementos como `<header>`, `<footer>`, `<main>`, `<section>`, `<article>`, entre outros, para descrever melhor o conteúdo. Usar essas tags ajuda na acessibilidade, no SEO (otimização para mecanismos de busca) e na compreensão geral do código.

Respeite a hierarquia dos títulos

Comece com `<h1>` para o título principal e siga com `<h2>`, `<h3>`... conforme a estrutura do conteúdo. Nunca pule níveis (por exemplo, não use `<h3>` diretamente após um `<h1>`). Isso facilita a navegação por leitores de tela e mantém o conteúdo organizado.

Comente trechos relevantes do código

Use comentários (`<!-- Comentário -->`) para marcar seções do código ou explicar partes complexas. Isso ajuda você e outras pessoas a entenderem melhor o que foi feito, especialmente em arquivos maiores ou projetos colaborativos.

Use nomes claros para IDs e classes

Ao nomear elementos com atributos `id` ou `class`, escolha nomes descritivos e específicos. Por exemplo, prefira `menu-principal` a simplesmente `box1`. Isso facilita a leitura e a manutenção do código, além de melhorar a integração com CSS e JavaScript.

Valide seu código

Antes de finalizar um documento HTML, use ferramentas de validação como o validador do W3C (<https://validator.w3.org/>). Isso ajuda a identificar erros de sintaxe e boas práticas que talvez tenham sido esquecidas.

7. Alternativas para Diagramar uma Página: Tabelas vs Divs

Ao desenvolver uma página web, é necessário organizar visualmente os elementos na tela. Esse processo é chamado de **diagramação**. Existem diferentes formas de fazer isso, sendo as mais comuns o uso de **tabelas (tables)** e de **divisões (divs)** com auxílio de CSS.

Uso de Tabelas para Diagramação

Antigamente, o uso de tabelas era a principal forma de organizar o layout de páginas. Por exemplo, criava-se uma tabela com várias células (`<td>`) e inseria-se o conteúdo dentro delas.

Vantagens:

- Simples de usar para layouts fixos
- Facilidade de posicionamento sem conhecimento de CSS

Desvantagens:

- Pouca flexibilidade
- Dificulta a responsividade (adaptação para celular e tablets)
- Código mais difícil de manter e menos semântico
- Não recomendado para layouts modernos

Uso de Divs para Diagramação

Hoje em dia, o padrão moderno é usar <div> (blocos genéricos) junto com CSS para estruturar e posicionar elementos. Com o CSS, podemos criar layouts responsivos, centralizar elementos, dividir colunas, controlar margens, tamanhos e muito mais.

Vantagens:

- Total controle sobre o visual usando CSS
- Suporte à responsividade (mobile first)
- Código mais limpo, semântico e organizado

Desvantagens:

- Requer conhecimento básico de CSS
- Um pouco mais complexo para iniciantes

Conclusão:

Para projetos atuais e profissionais, o uso de <div> com CSS é a abordagem recomendada. O uso de tabelas deve ser reservado para exibição de dados tabulares (como planilhas), e não para layout da página.